

AADK 4 – Task 1 - Composables Practice

Reflection & Planning

Student Name: Rajat Prakash Dhal

Course/Unit: Android Development with Kotlin – Session 4

USN: 1hk22cs115

email: 1hk22cs115@hkbk.edu.in

1. Introduction

This report reflects on selected exercises from the “Practice: Compose Basics” codelab. The purpose of this reflection is to analyse how these problems improved my understanding of composables such as Row, Column, and Box, and how planning before coding supports better UI development in Jetpack Compose.

2. Problem Review and Analysis

2.1 Problem 1: Vertical Greeting Layout (Column + Text)

Problem Summary:

The exercise required building a simple screen displaying a greeting message and a short description arranged vertically. The layout needed proper alignment and spacing using composables.

Planned Approach (Before Coding):

Before implementation, I planned to:

- Use Column as the root composable to stack elements vertically.
- Add two Text composables (title and description).
- Apply Modifier.fillMaxSize() and padding(16.dp).
- Use verticalArrangement and horizontalAlignment to control positioning.

Actual Implementation & Reflection:

The layout functioned correctly using Column. However, I initially overlooked padding, which caused the UI to appear crowded. I also needed better understanding of alignment properties within Column. In future, I would sketch spacing and alignment before writing code to avoid trial-and-error adjustments.

2.2 Problem 2: Horizontal Layout with Equal Items (Row + Box/Text)

Problem Summary:

This task required building a horizontal layout with multiple elements arranged side-by-side using Row. The elements needed equal spacing and consistent alignment.

Planned Approach (Before Coding):

I planned to:

- Use Row with Modifier.fillMaxWidth().
- Insert multiple Text or Box elements.
- Apply horizontalArrangement = Arrangement.SpaceEvenly.
- Use Modifier.weight(1f) for equal distribution if needed.

Actual Implementation & Reflection:

The layout worked after applying Arrangement.SpaceEvenly. The challenging part was understanding how weight() distributes space and how it interacts with parent layout constraints. I learned that layout behavior changes depending on screen width. Next time, I would test responsiveness earlier in the process.

3. Planning Checklist Template for Future UI Problems

When approaching a new Compose UI problem, I will follow this structured process:

1. Understand the Requirements

- Identify layout direction (vertical, horizontal, layered).
- Note alignment and spacing needs.

2. Create a Rough Sketch

- Draw a simple wireframe.
- Identify layout containers and child elements.

3. Select Appropriate Composables

- Column (vertical structure)
- Row (horizontal structure)
- Box (layered content)

- Text, Image, Button as required

4. Plan Modifiers

- Padding values (8 dp, 16 dp, etc.)
- `fillMaxWidth()` or `wrapContentSize()`
- Alignment and arrangement
- `weight()` if equal distribution is needed

5. Test and Review

- Check on multiple screen sizes.
- Ensure spacing consistency.
- Simplify code where possible.

4. Conclusion and Reflection

Planning and reflection significantly improve my ability to work with Compose UIs. Instead of randomly placing composables, I can structure layouts logically. Reflection helps identify common mistakes such as missing padding or incorrect alignment. Over time, this approach builds confidence and efficiency in designing responsive and clean user interfaces in Jetpack Compose.